

Group#_Phase1

Project Title: Multi-Tenant LLM API Gateway with Billing, Quota, and Audit

Group: abc

Submitted by:

- Member 1: runxin shao
 - Member 2: ziqi yang
-

Abstract

As organizations adopt Large Language Models (LLMs) through APIs, they face recurring challenges in cost governance, quota control, reliability, and compliance auditing—especially in multi-team or multi-tenant settings. This project proposes a Multi-Tenant LLM API Gateway that acts as a unified entry point for LLM calls. The gateway handles authentication (API keys), quota enforcement, billing, rate-limit logging, request/response tracing, and audit queries.

The project is intentionally database-centric: a relational database is required to guarantee transactional consistency for quota deduction under concurrent calls, enforce referential integrity across tenants/projects/keys/models, and support structured analytical reporting (cost summaries, error breakdowns, anomaly detection, and audit trails). The backend will use MySQL (preferably on GCP Compute Engine; local MySQL acceptable). The LLM itself can be mocked; the main focus is schema design, SQL querying, and transaction logic.

Step 1: Project Information

1.1 Team

This is a group project with 3–4 members.

1.2 Technology Stack

- RDBMS (Back-end Storage): MySQL
 - Preferred: MySQL deployed on GCP Compute Engine
 - Acceptable: local MySQL for final demo
- Application Layer (API): Web-based system
 - Recommended frameworks: FastAPI or Django (Python)

- Client Layer: Simple web UI / CLI / scripts to send requests
- LLM Execution: Not required (mock responses are acceptable)

1.3 High-Level Architecture

1. Client sends request to API Gateway endpoint with an API key
 2. Gateway authenticates the key and identifies tenant/project
 3. Gateway checks quota and updates usage transactionally
 4. Gateway records request and response metadata
 5. Analysts/admins query reports, invoices, and audits from MySQL
-

Step 2: Problem Statement

2.1 Project Overview

The system provides a centralized gateway for LLM API usage across multiple tenants (teams) and projects. Instead of direct calls to an LLM provider, all requests flow through the gateway so that the system can consistently apply governance and logging policies.

Core system capabilities:

- API key management (issue/revoke/rotate, status tracking)
- Monthly quota definition and enforcement
- Billing and cost attribution (per tenant/project/model/key)
- Request/response tracing (tokens, latency, status, error type)
- Rate-limit / denied request logging
- Audit queries for compliance investigations
- Reporting and analytics (daily trends, top spenders, failures)

2.2 Problem Definition

Without a unified gateway and a structured database, organizations often experience:

- Inaccurate quota enforcement under concurrency: multiple parallel requests can cause overspending if quota updates are not transactional.
- Weak traceability: scattered logs make it hard to answer “who used which key to

call which model at what time, with what outcome and cost”.

- Cost governance difficulties: teams need aggregated monthly/daily cost reporting and attribution by tenant/project/model.
- Compliance risks: revoked keys or abnormal usage patterns must be detected through structured queries.

These requirements motivate a relational database solution because they depend on transactions, locking, integrity constraints, and analytical SQL queries.

2.3 Target Users

- Platform Admin: manages tenants/projects, API keys, model catalog, pricing, and quotas.
- API Client (Project User): calls the gateway using API keys and wants reliable access and usage tracking.
- Finance Analyst: monitors spending and generates invoices.
- Security/Auditor: runs audit/compliance investigations.

2.4 Project Scope

In Phase I, we define the problem, scope, actors, and query requirements. The system scope includes:

- Multi-tenant structure (tenant → projects → API keys)
- Model catalog and pricing configuration
- Request logging and response/error tracking
- Monthly quotas and usage aggregation
- Analytics and audit query capabilities

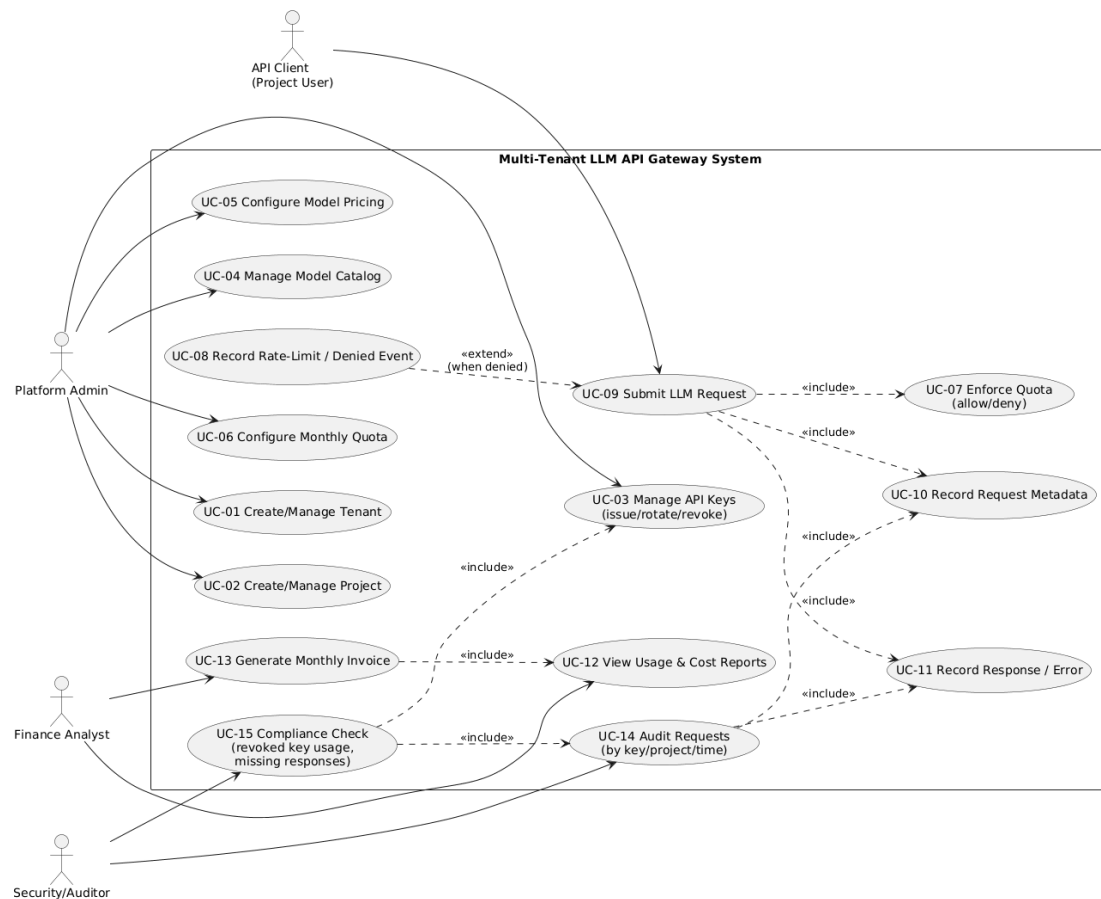
Implementation scope (future phases):

- MySQL schema + constraints
- API endpoints (create keys, submit request, view reports)
- Transactional quota deduction
- Reports and invoice generation

2.5 Real-World Relevance

This system reflects real practices in AI infrastructure and LLMOps, where organizations must control LLM costs, ensure reliability, and maintain audit trails. Database-driven quota and billing systems are widely used for governance and compliance in production environments.

Step 2 (Model Tool):



UML Use Case Model

Actors

- API Client (Project User)
- Platform Admin
- Finance Analyst
- Security/Auditor

Use Cases

Administration

- UC-01 Create/Manage Tenant
- UC-02 Create/Manage Project
- UC-03 Manage API Keys (issue/rotate/revoke)

Configuration

- UC-04 Manage Model Catalog
- UC-05 Configure Model Pricing
- UC-06 Configure Monthly Quota

Runtime & Logging

- UC-07 Enforce Quota (allow/deny)
- UC-08 Record Rate-Limit / Denied Event
- UC-09 Submit LLM Request
- UC-10 Record Request Metadata
- UC-11 Record Response / Error

Reporting & Audit

- UC-12 View Usage & Cost Reports
- UC-13 Generate Monthly Invoice
- UC-14 Audit Requests (by key/project/time)
- UC-15 Compliance Check (revoked key usage, missing responses)

Include / Extend Relations

- UC-09 includes UC-10, UC-07, UC-11
- UC-08 extends UC-09 (only when denied/rate-limited)
- UC-13 includes UC-12
- UC-14 includes UC-10 and UC-11
- UC-15 includes UC-14 and UC-03

Step 3: At Least 15 Realistic Queries (English)

1. For each project, show total cost and total tokens (input/output) over the last 30 days.

2. Aggregate monthly cost by tenant and list the top 5 most expensive projects per tenant.
3. For each model, show success rate, average latency, and failure rate over the last month.
4. Breakdown failed requests by error_type per model and per project.
5. Detect projects with request volume in the last 24 hours significantly higher than their 7-day average.
6. Audit query: list all requests made by a specific API key between two timestamps.
7. Find duplicate idempotency_key values within a project (potential replay/retry duplicates).
8. List projects that have used more than 80% of their monthly quota.
9. Top 10 API keys with the most rate-limit/denied events in the last 30 days.
10. Identify projects that continue high-frequency calls after crossing a quota threshold (e.g., 90%).
11. Compare usage and cost across versions of the same model (by provider/name/version).
12. Find projects with unusually high output_tokens / input_tokens ratio.
13. List the 100 slowest requests with their model, project, and latency_ms.
14. Compare cost and failure rate between environments (dev vs prod).
15. Find requests that have no matching response record (missing response anomaly).
16. Cost attribution: split total cost by API key within the same project for the current month.
17. Daily cost time series for a given tenant or project for the past 90 days.
18. Identify projects where an invoice exists for the month but usage is zero (data inconsistency check).
19. Compliance check: list any requests made by revoked API keys after revoked_at time.
20. Identify the most expensive tenants and the models contributing most to their

spending.

Step 4: Assumptions

1. Unique Identifiers: Tenants, projects, API keys, models, requests, and bills/invoices have unique identifiers (surrogate primary keys are allowed).
 2. Tenant–Project Structure: A tenant can own multiple projects; each project belongs to exactly one tenant.
 3. API Key Ownership: Each API key belongs to exactly one project. An API key can be active or revoked.
 4. Model Pricing: Each model has stable pricing fields (e.g., cost per 1K input/output tokens) during a billing month.
 5. One Response per Request: Each request generates at most one response record (success or failure).
 6. Monthly Quotas: Quotas are defined per project per month (YYYY-MM).
 7. Transactional Quota Enforcement: Quota usage updates must be performed in a database transaction to prevent overspending under concurrent requests.
 8. Audit Integrity: Foreign key constraints are enforced to preserve traceability from request → key → project → tenant and request → response.
 9. Denied Requests: Rate-limited or quota-denied attempts can still be recorded for analysis.
 10. LLM Execution is Mocked: The system may use a mock service to return output_tokens, latency_ms, and status codes; the database and transaction logic remain the project focus.
-

Conclusion

This Phase I work defined a realistic and database-driven DBMS project: a multi-tenant LLM API gateway supporting quota enforcement, billing, and auditability. The problem is strongly tied to relational database features such as transactions, referential integrity, and complex analytical queries. In researching and shaping the idea, we learned how system requirements (cost governance, reliability, and compliance) directly translate into database schema constraints and query workloads.

To achieve the best final output, the next phase will require:

- A normalized MySQL schema with correct keys and constraints,
- Implementation of transactional quota enforcement,
- And validated SQL queries for reporting, invoicing, and auditing.